



MA 725 : 25 - OCT - 2022

Data Driven Reduced-Order Modeling Using Neural Networks

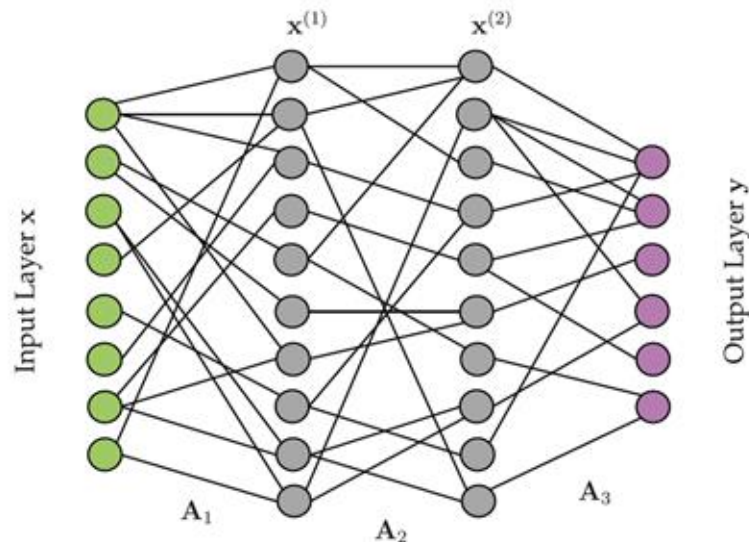
Rammini Nipuni Senani de Silva

Department of Mathematics
Clarkson University, Potsdam
desilvr@clarkson.edu

Neural Network : Definition

A neural network is a **method in artificial intelligence that teaches computers to process data in a way that is inspired by the human brain.**

It is a type of machine learning process, called deep learning, that uses interconnected nodes or neurons in a layered structure that resembles the human brain.



$x^{(1)}, x^{(2)}$: Layers

$A_1 : A_3$: mappings

$w_{ij}^{(k)}$: Weights of A_k th mapping

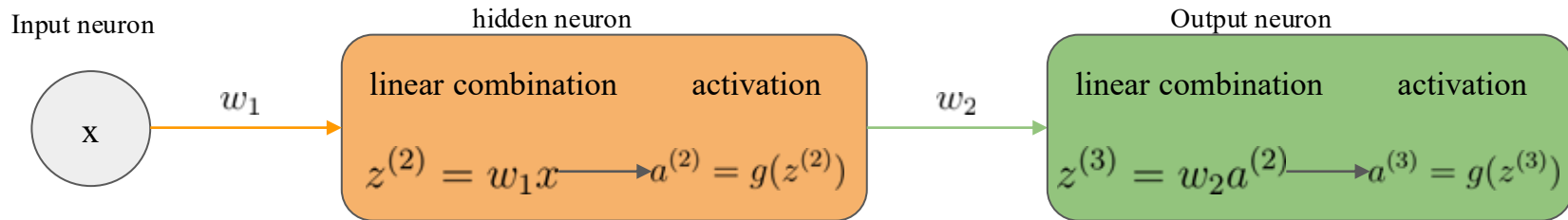
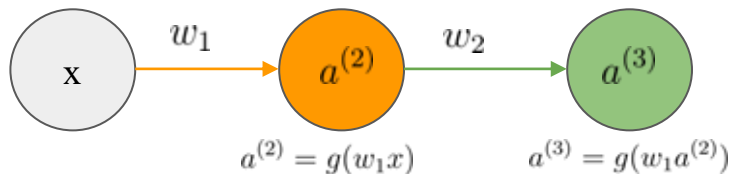
Mapping can be linear or nonlinear

$$x^{(i+1)} = A_i x^{(i)}$$

$$y = A_M A_{M-1} \dots A_2 A_1 x$$

Simple Neural network

The simplest neural network: one input neuron, one hidden layer neuron, and one output neuron.



Neural Network: Train

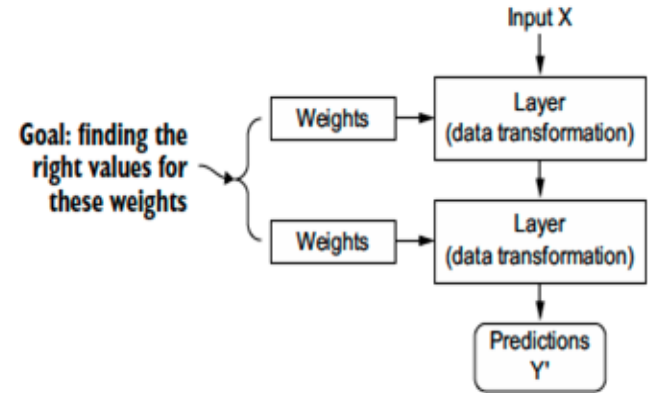
Training data : The data builds up the learning algorithm.

Validation data: During training, validation data infuses new data into the model that it hasn't evaluated before. Validation data provides the first test against unseen data, allowing to evaluate how well the model makes predictions based on the new data.

Test data: After the model is built, testing data once again validates that it can make accurate predictions.

Batch size : a number of samples processed before the model is updated.

The number of epochs : the number of complete passes through the training dataset.



My Project

- Build a Neural Network model for a given dynamical system .
- using Autoencoders and Neural Ordinary Differential Equations.

Linot, Alec J., and Michael D. Graham. "Data-driven reduced-order modeling of spatiotemporal chaos with neural ordinary differential equations." *Chaos: An Interdisciplinary Journal of Nonlinear Science* 32.7 (2022): 073110.

What I have done so far

Project contains three parts...



1. Neural Network model to obtain reduce dimensional manifold data of the dynamical system



1. Neural Network model that mapped corresponding ODE model of the dynamical system data



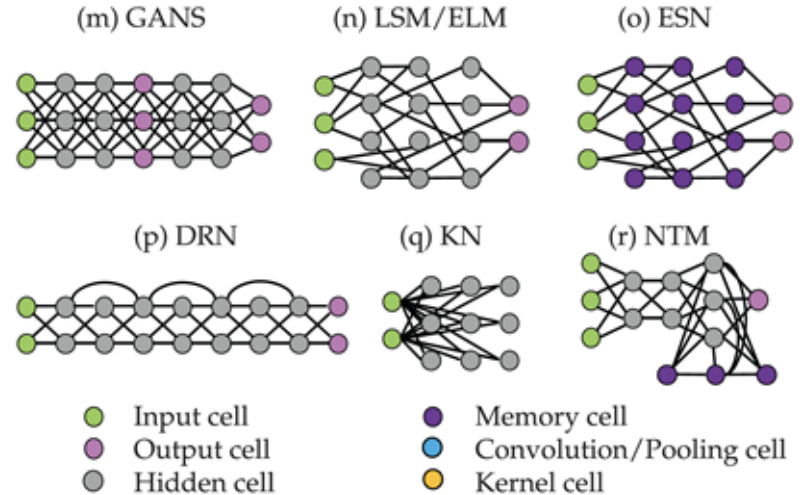
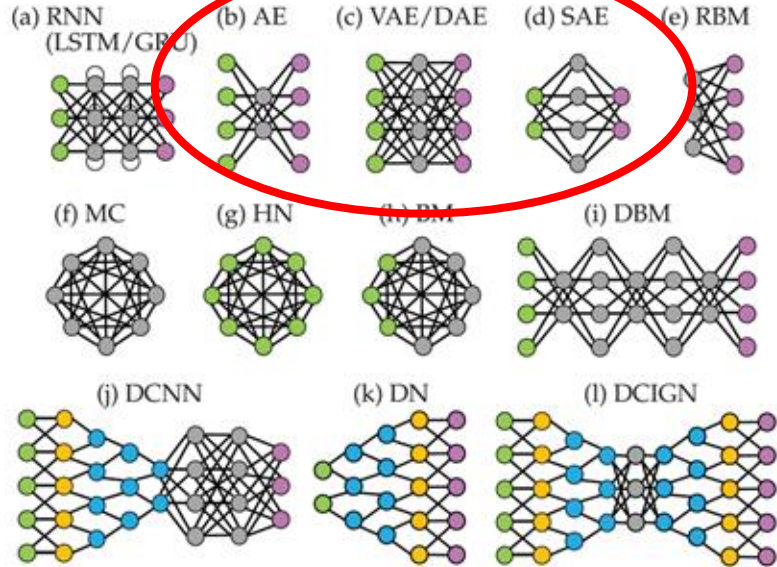
1. By using part 1 and part 2 , predict the future time step values of the given system

Part 1: Intro

1. Neural Network model to obtain reduce dimensional manifold data of the dynamical system



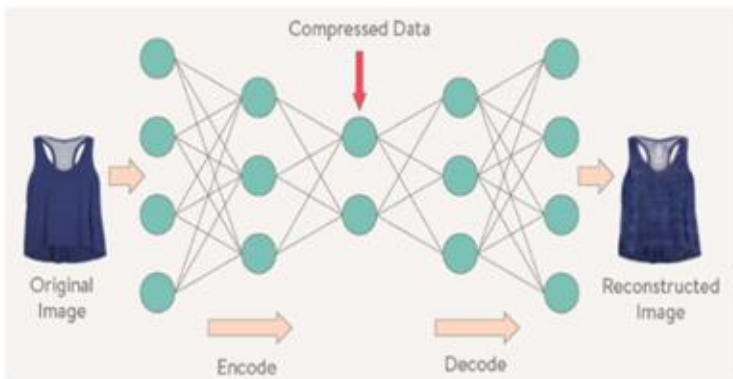
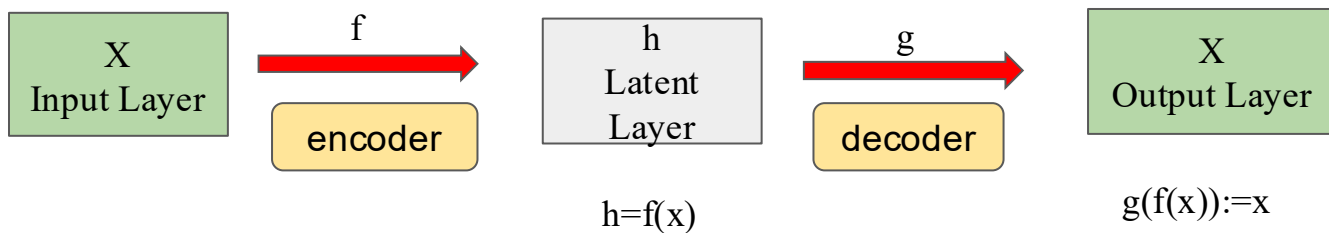
Part 1: Neural network types



Part 1: Autoencoders

Autoencoder

An autoencoder is a neural network that is trained to attempt to copy its input to its output. Internally It has a hidden layer h that describes a code used to represent the input.

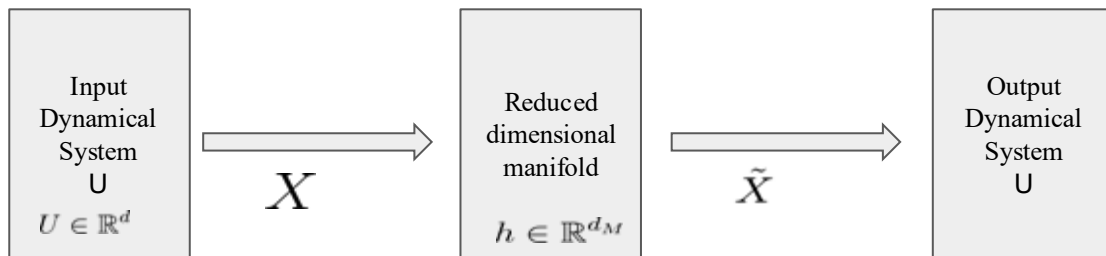


Undercomplete Autoencoder

Undercomplete Autoencoders

An autoencoder whose code dimension(latent layer dimension) is less than the input dimension is called the undercomplete autoencoder.

- Constrain h to have lower dimension than U
- Force it to capture most salient features of training data



Ambient space

Where h is the state representation in manifold coordinates

Autoencoder: Simulations

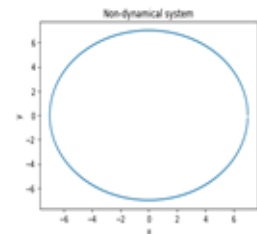
Data Set:

- Taken from a single trajectory of each system
- Evenly spaced points in the trajectory

Inputs of NN:

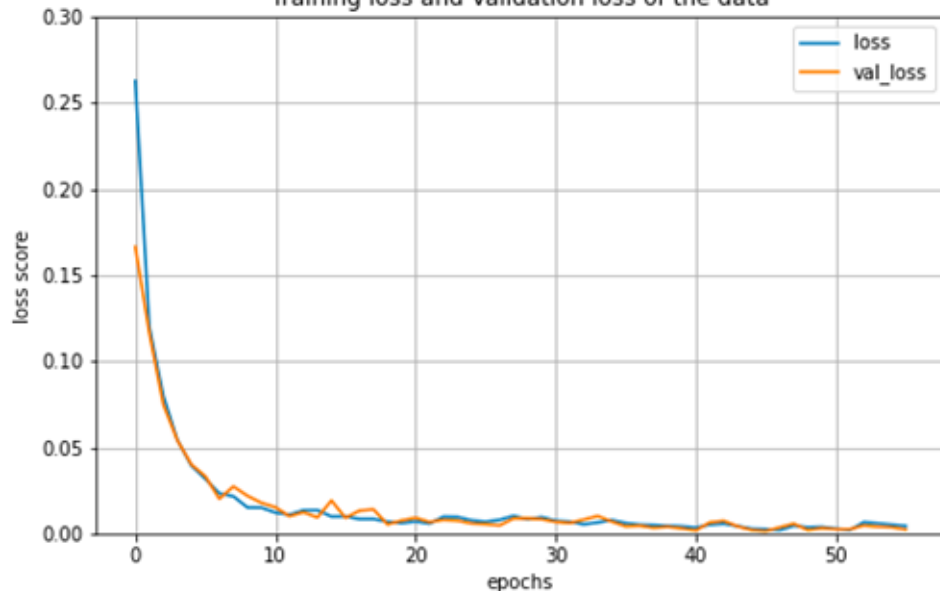
- Considered all state variables of each data point
- Data points are randomly selected from data set
- Training set : 70% data set
- Validation set: 15% data set
- Test set : 15% data set

Autoencoder: Circle $2D \rightarrow 1D \rightarrow 2D$



$$\begin{aligned} x &= r \cos(\theta) \\ y &= r \sin(\theta) \end{aligned} \iff \theta = \arctan\left(\frac{y}{x}\right)$$

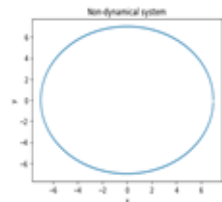
Training loss and Validation loss of the data



Model: "autoencoder"

Layer (type)	Output Shape	Param #
=====		
cir (InputLayer)	[(None, 2)]	0
flatten (Flatten)	(None, 2)	0
dense (Dense)	(None, 16)	48
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 4)	36
dense_3 (Dense)	(None, 1)	5
dense_4 (Dense)	(None, 4)	8
dense_5 (Dense)	(None, 8)	40
dense_6 (Dense)	(None, 16)	144
dense_7 (Dense)	(None, 2)	34
reshape (Reshape)	(None, 2)	0
=====		
Total params: 451		
Trainable params: 451		
Non-trainable params: 0		

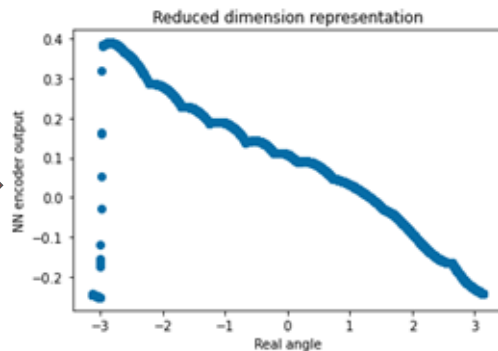
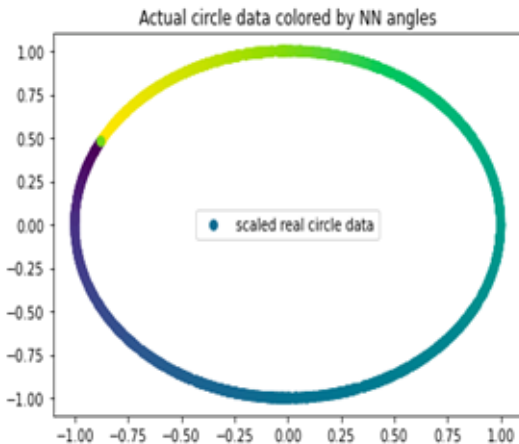
Autoencoder: Circle $2D \rightarrow 1D \rightarrow 2D$



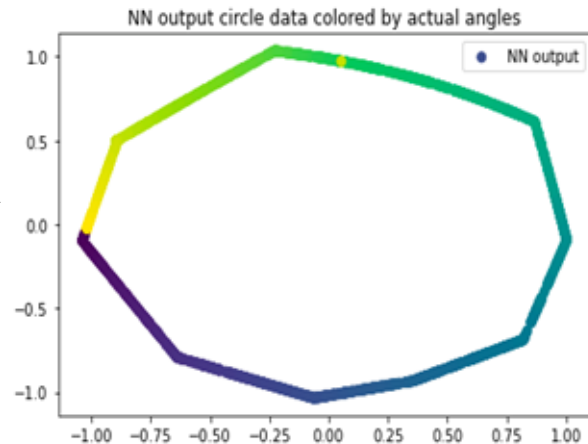
$$x = r \cos(\theta)$$

$$y = r \sin(\theta)$$

Data set : 10000 points
Loss function : Mean Square Error
Optimizer: Adam



$$\theta = \arctan\left(\frac{y}{x}\right)$$

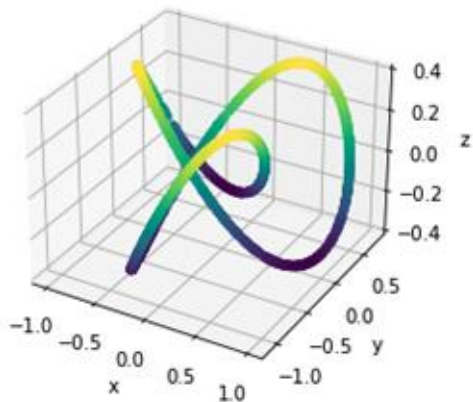


Autoencoder: Trefoil Knot $3D \rightarrow 2D \rightarrow 3D$

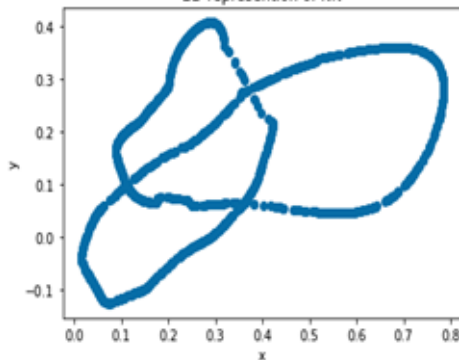


$$\begin{aligned}x &= \sin t + 2 \sin 2t \\y &= \cos t - 2 \cos 2t \\z &= -\sin 3t\end{aligned}$$

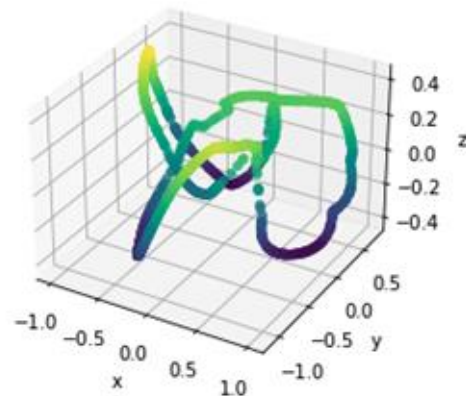
Test data :trefoil knot



2D representation of NN

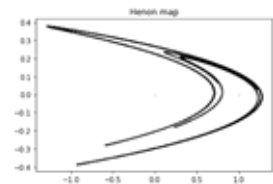


NN output :trefoil knot

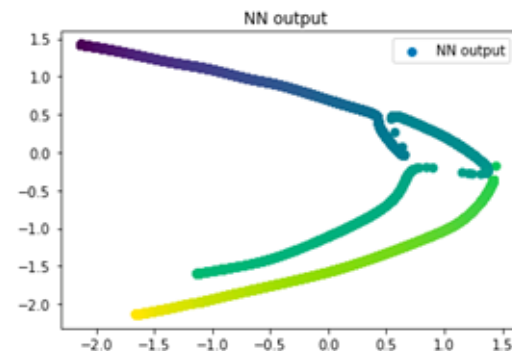
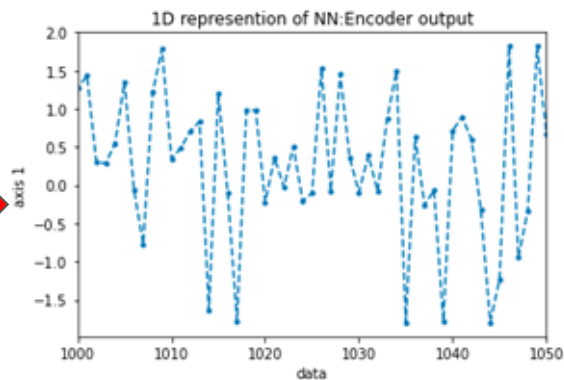
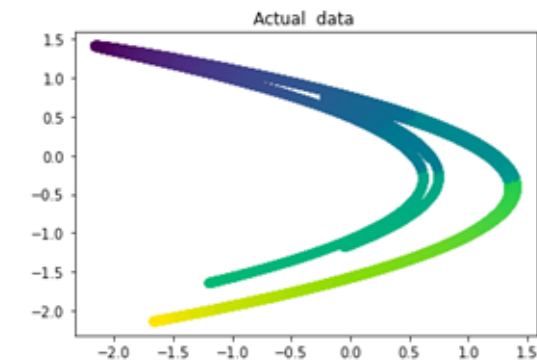


Part 1:Hénon Map

2D → 1D → 2D



$$\begin{aligned}x_{n+1} &= 1 - ax_n^2 + y_n \\y_{n+1} &= bx_n \\a &= 1.4 \\b &= 0.3\end{aligned}$$



Autoencoder: Lorenz system $4D \rightarrow 3D \rightarrow 4D$

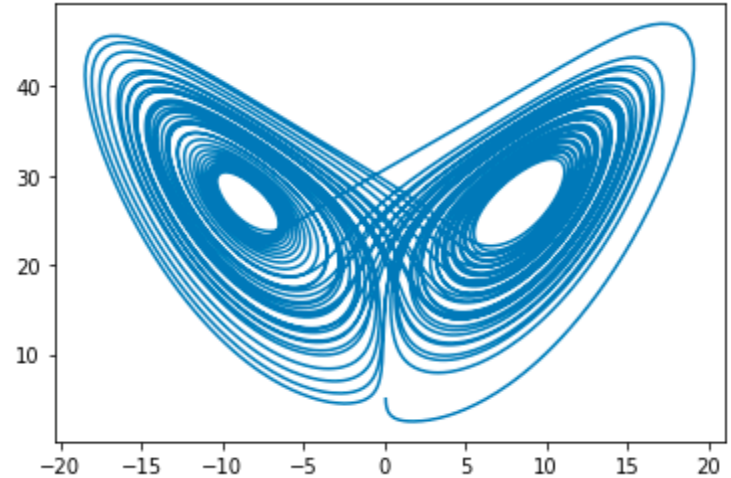


Original representation(3D):

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x) \\ \frac{dy}{dt} &= x(\rho - z) - y \\ \frac{dz}{dt} &= xy - \beta z\end{aligned}$$

Four Dimension representation:

$$[u_1, u_2, u_3, u_4] = [x, y, \alpha z \cos(\alpha z), \alpha z \sin(\alpha z)]$$



Data set : 10000 points

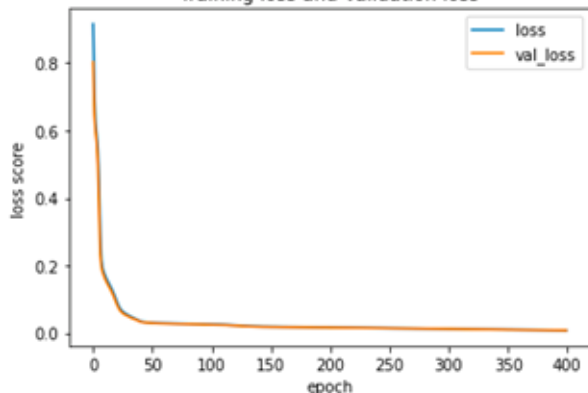
Batch : 32

Loss function : Mean Square Error

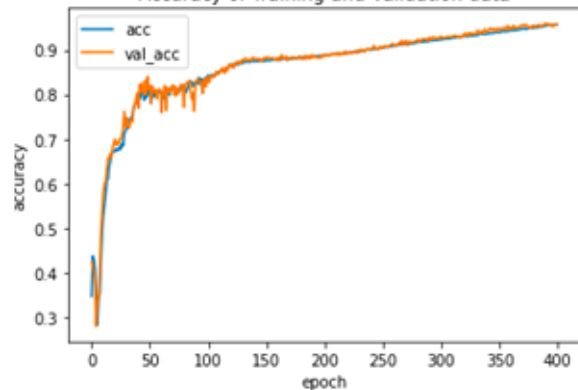
Optimizer: Adam

Autoencoder: Lorenz system $4D \rightarrow 3D \rightarrow 4D$

Training loss and Validation loss



Accuracy of Training and Validation data



Model: "autoencoder"

Layer (type)	Output Shape	Param #
Lorenz (InputLayer)	[(None, 4)]	0
flatten_7 (Flatten)	(None, 4)	0
dense_92 (Dense)	(None, 4)	20
dense_93 (Dense)	(None, 3)	15
dense_94 (Dense)	(None, 4)	16
dense_95 (Dense)	(None, 4)	20
reshape_7 (Reshape)	(None, 4)	0

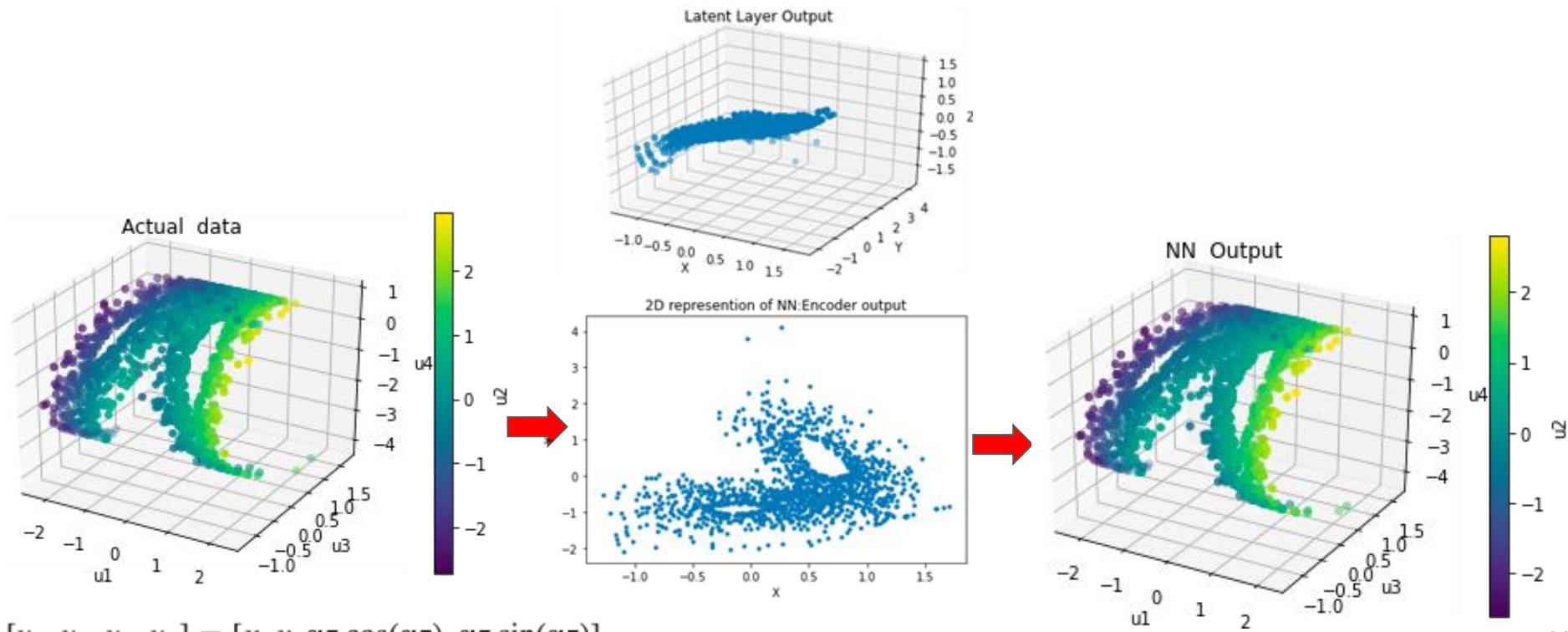
Total params: 71

Trainable params: 71

Non-trainable params: 0

Part 1: Lorenz system

4D $\rightarrow$ 3D $\rightarrow$ 4D



$$[u_1, u_2, u_3, u_4] = [x, y, \alpha z \cos(\alpha z), \alpha z \sin(\alpha z)]$$

Conclusion

- Neural networks are one of the best ways to learn features of data with unknown models.
- Our aim here is to create a neural network to predict future state of a given dynamical system.
- Dimension reduction of the data is important along the process.
- Autoencoders can learn mapping back and forth between high dimensional data and reduce dimensional data.
- We use autoencoders to reduce the dimension of the given system data before we create the corresponding N-ODE model of the system.

Thank You!